Predicting Game Ratings on Steam

Question

Can the rating of a game be predicted based on its metadata (such as genres, price, and categories)? I'll be predicting the variable `pct_pos_total`, which takes on values from 0 to 100 and represents the percentage of positive reviews that the game has. Answering this question can be useful for game developers to estimate how well their game might perform on Steam. If metadata can predict future user ratings, then developers can use higher-performing metadata if they want their game to have higher positive ratings.

Data and Setup

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ dplyr     1.1.4     ✓ readr     2.1.6
## ✓ forcats   1.0.1     ✓ stringr   1.6.0
## ✓ ggplot2   4.0.1     ✓ tibble    3.3.1
## ✓ lubridate 1.9.4     ✓ tidyr     1.3.2
## ✓ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
e errors
```

```
library(tidymodels)
```

```
## ── Attaching packages ─────────────────────────────────────── tidymodels 1.4.1 ──
## ✓ broom        1.0.11     ✓ rsample      1.3.2
## ✓ dials        1.4.2      ✓ tailor       0.1.0
## ✓ infer        1.1.0      ✓ tune         2.0.1
## ✓ modeldata    1.5.1      ✓ workflows    1.3.0
## ✓ parsnip      1.4.1      ✓ workflowsets 1.1.1
## ✓ recipes      1.3.2      ✓ yardstick    1.3.2
## ── Conflicts ────────────────────────────────────────── tidymodels_conflicts() ──
## ✗ scales::discard() masks purrr::discard()
## ✗ dplyr::filter()   masks stats::filter()
## ✗ recipes::fixed()  masks stringr::fixed()
## ✗ dplyr::lag()      masks stats::lag()
## ✗ yardstick::spec() masks readr::spec()
## ✗ recipes::step()   masks stats::step()
```

```r
library(stringr)
library(textrecipes)
library(ranger)
set.seed(123)

## Add your code for reading in the data here
games <- read_csv("games_march2025_cleaned.csv")
```

```
## Rows: 89618 Columns: 47
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr  (22): name, detailed_description, about_the_game, short_description, re...
## dbl  (21): appid, required_age, price, dlc_count, metacritic_score, achievem...
## lgl   (3): windows, mac, linux
## date  (1): release_date
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
## Removes games with invalid/no reviews and no playtime
games <- games |>
  filter((positive + negative) > 0, pct_pos_total > 0, peak_ccu > 0) |>

  ## Remove brackets from genres and categories columns
  mutate(categories = str_remove_all(categories, "[\\[\\]']"),
         genres = str_remove_all(genres, "[\\[\\]']")) |>

  ## Keep only relevant variables
  select(name, required_age, price, dlc_count, achievements, recommendations, categories, genre
s, peak_ccu, positive, negative, estimated_owners, pct_pos_total)
```

Base Prediction Model

```
## Splits the dataset into training and testing datasets
games_split <- initial_split(games, prop = 0.75, strata = pct_pos_total)
games_train <- training(games_split)
games_test  <- testing(games_split)

## Define recipe and select variables (price, genres, categories, peak concurrent connected user
s)
game_rec <- games_train |>
  recipe(pct_pos_total ~ price + genres + categories + peak_ccu) |>
  step_normalize(price, peak_ccu) |>
  step_tokenize(genres, categories) |>
  step_tokenfilter(genres, max_tokens = 8) |> ## Keep only the most popular genres
  step_tokenfilter(categories, max_tokens = 4) |> ## Keep only the most popular categories
  step_tfidf(genres, categories)

model <- linear_reg() |>
  set_engine("lm") |>
  set_mode("regression")

wf <- workflow() |>
  add_recipe(game_rec) |>
  add_model(model)

## Use 10-fold cross-validation to estimate the RMSE
game_folds <- vfold_cv(games_train, v = 10)
model_fit <- fit_resamples(wf, resamples = game_folds)
model_fit |>
  collect_metrics()
```

```
## # A tibble: 2 × 6
##   .metric .estimator    mean     n std_err .config
##   <chr>   <chr>        <dbl> <int>   <dbl> <chr>
## 1 rmse    standard   13.3       10 0.0857  pre0_mod0_post0
## 2 rsq     standard    0.0750    10 0.00447 pre0_mod0_post0
```

Price + genres + categories + peak_ccu seems to produce the best performing model on the training data with an RMSE of about 13.2949 and an RSQ of about 0.07500.

Alternate Prediction Model

```
## Identifies the tuning parameters for random forest
rf_model <- rand_forest(
  mtry = tune(),
  trees = 500,
  min_n = tune()
) |>
  set_engine("ranger") |>
  set_mode("regression")

rf_wf <- workflow() |>
  add_recipe(game_rec) |>
  add_model(rf_model)

## Tune model using grid
rf_grid <- grid_regular(
  mtry(range = c(2, 8)),
  min_n(range = c(5, 20)),
  levels = 3
)

rf_res <- tune_grid(
  rf_wf,
  resamples = game_folds,
  grid = rf_grid
)

## Finds the performance metrics of the random forest model
rf_res |>
  collect_metrics() |>
  filter(.metric == "rmse") |>
  arrange(mean)
```

```
## # A tibble: 9 × 8
##    mtry min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1     2    20 rmse    standard    12.8    10  0.0913 pre0_mod3_post0
## 2     2    12 rmse    standard    12.8    10  0.0891 pre0_mod2_post0
## 3     2     5 rmse    standard    12.9    10  0.0913 pre0_mod1_post0
## 4     5    20 rmse    standard    12.9    10  0.0951 pre0_mod6_post0
## 5     8    20 rmse    standard    12.9    10  0.0920 pre0_mod9_post0
## 6     5    12 rmse    standard    12.9    10  0.0921 pre0_mod5_post0
## 7     8    12 rmse    standard    13.0    10  0.0944 pre0_mod8_post0
## 8     5     5 rmse    standard    13.1    10  0.0957 pre0_mod4_post0
## 9     8     5 rmse    standard    13.2    10  0.0953 pre0_mod7_post0
```

```
rf_res |>
  collect_metrics() |>
  filter(.metric == "rsq") |>
  arrange(desc(mean))
```

```
## # A tibble: 9 × 8
##    mtry min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1     2    20 rsq     standard   0.143    10 0.00632 pre0_mod3_post0
## 2     2    12 rsq     standard   0.140    10 0.00623 pre0_mod2_post0
## 3     5    20 rsq     standard   0.138    10 0.00706 pre0_mod6_post0
## 4     8    20 rsq     standard   0.136    10 0.00660 pre0_mod9_post0
## 5     2     5 rsq     standard   0.136    10 0.00619 pre0_mod1_post0
## 6     5    12 rsq     standard   0.131    10 0.00615 pre0_mod5_post0
## 7     8    12 rsq     standard   0.127    10 0.00635 pre0_mod8_post0
## 8     5     5 rsq     standard   0.121    10 0.00613 pre0_mod4_post0
## 9     8     5 rsq     standard   0.114    10 0.00580 pre0_mod7_post0
```

The random forest model performed best with an mtry of 2 and a min_n of 20. This combination achieved an RMSE of about 12.8028 and R-squared of about 0.1432, which has better performance than the base linear regression model that had an RMSE of about 13.2949 and R-squared of about 0.07500. The Random Forest likely performed better because it can capture non-linear relationships between variables like price and peak_ccu that the linear regression model might miss.

Final Model Fit

```
## Select the best parameters
best_rf <- rf_res |>
  select_best(metric = "rmse")

final_wf <- rf_wf |>
  finalize_workflow(best_rf)

## Fit to the full training set and run on test set
final_fit <- final_wf |>
  last_fit(games_split)

final_fit |>
  collect_metrics()
```

```
## # A tibble: 2 × 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard      12.8   pre0_mod0_post0
## 2 rsq     standard       0.147 pre0_mod0_post0
```

After selecting the best tuning parameters and variables and fitting the model to the testing dataset, the final RMSE was about 12.7777 and the R-squared was about 0.1468 These metrics suggest that while random forest model can explain some of the variance in game ratings, there are many other factors influencing positive review

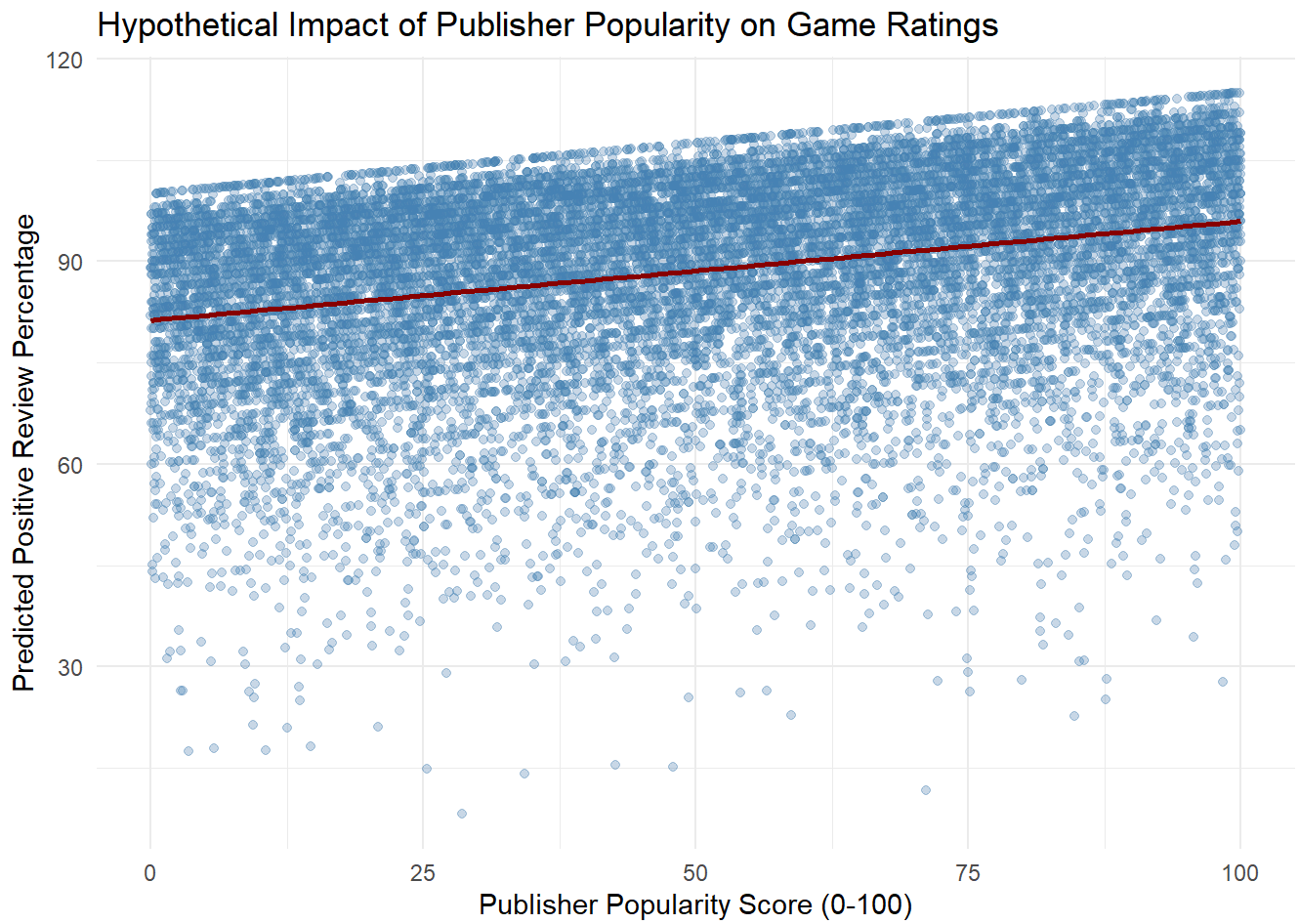
percentage that aren’t captured by price or genre alone.

Model Improvement

```
## Simulates the hypothetical relationship between publisher popularity and positive review perc
entage
games_publisher_sim <- games |>
  # Creates a simulated popularity score (0 = Unknown, 100 = AAA)
  mutate(publisher_popularity = runif(n(), 0, 100)) |>
  # Simulates that popular publishers have a higher floor for ratings
  mutate(hypothetical_rating = pct_pos_total + (publisher_popularity * 0.15))

## Plot hypothetical visualization
games_publisher_sim |>
  ggplot(aes(x = publisher_popularity, y = hypothetical_rating)) +
  geom_point(alpha = 0.3, color = "steelblue") +
  geom_smooth(method = "lm", color = "darkred") +
  theme_minimal() +
  labs(
    title = "Hypothetical Impact of Publisher Popularity on Game Ratings",
    x = "Publisher Popularity Score (0-100)",
    y = "Predicted Positive Review Percentage"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



I would most like to collect data on game publisher popularity. Currently, the random forest model only sees what the game is and its metadata. A game from a well-known publisher likely receives a higher rating due to established trust and higher production budgets, which are not currently captured by price or genre. This could increase the R-squared by explaining the portion of positive review percentages that are connected to brand reputation rather than just the game's individual attributes.

Discussion

I thought that handling the data was challenging. Cleaning the dataset was tedious since most of the variables were given in formats that would be hard to work with, so I had to recode them. Finding the combination of variables and tuning parameters to find the best performing model was also pretty challenging. I learned how to implement a full machine learning model using the tidymodels ecosystem. More specifically, I learned about how to tune the model and find the best performing configuration. I was surprised by how low the R-squared values were across both the linear and machine learning models (around 0.12-0.15). I initially expected that if a game had the "right" genres and the "right" price, it would naturally have a high rating. However, the results showed that these variables only explain a small portion of the variance in user reviews.This suggests that on Steam, a game's success is influenced more by other factors like art style and how well the game was created.